Data 201

Intro to Data Analysis in Python

Visualization in Python for R Users

Week 2-3

Dr. Rebin Muhammad

Welcome — Week 3

Weeks 1–2 gave you Python fundamentals and pandas. This week we add **visualization**: the same “grammar of graphics” thinking you know from ggplot2, now in Python with seaborn and matplotlib.

Today (75 min): the Python viz stack, ggplot2→seaborn mapping, common plot types, figure control, and hands-on translation.

Have the **Week 3 Notebook** and your **R→Python cheat sheet** open.

What we'll do today (Week 3)

1. **The Python visualization stack** — matplotlib, seaborn, and how they fit together
2. **ggplot2 vs seaborn** — Same ideas; different syntax (aes → arguments, layers → function calls)
3. **Data first** — Tidy data from pandas feeds straight into plotting
4. **Common plots** — Scatter, histogram/density, boxplot/violin, faceting
5. **Figure control** — Titles, labels, limits, saving (matplotlib)
6. **Practice** — Translate a ggplot into seaborn and customize it

Learning goals

By the end of today you will be able to:

1. **Explain** how Python visualization libraries fit together (matplotlib vs seaborn)
2. **Create** common plots using seaborn (scatter, histogram, boxplot, facet)
3. **Customize** plots using matplotlib (titles, labels, axis limits, saving)
4. **Translate** ggplot2 concepts into Python

The Python visualization stack

How plotting works in Python:

- **matplotlib** — low-level engine: axes, labels, colors, saving. Everything eventually uses it.
- **seaborn** — high-level statistical graphics: concise API, sensible defaults, works on DataFrames.
- **pandas** — quick exploratory plots (e.g. `df.plot()`); often we use seaborn for anything we want to polish.

Think of seaborn as: ggplot-like, but built on top of matplotlib.

ggplot2 vs seaborn (big picture)

ggplot2	seaborn
Grammar of graphics	Grammar-inspired
`aes(x, y, color=...)`	Function arguments: `x="x", y="y", hue="..."`
Layers via +	One function per plot type; overlay by calling another
`geom_point()`, `geom_line()`	`sns.scatterplot()`, `sns.lineplot()`

Core ideas still transfer: data, aesthetics (mapping variables to visual properties), and geoms (plot types).

Quick reference: ggplot2 → seaborn

ggplot2	seaborn
`geom_point()`	`sns.scatterplot()`
`geom_line()`	`sns.lineplot()`
`geom_histogram()`	`sns.histplot()`
`geom_density()`	`sns.kdeplot()`
`geom_boxplot()`	`sns.boxplot()`
`geom_violin()`	`sns.violinplot()`
`facet_wrap(~ x)`	`sns.relplot(..., col="x")`
`aes(color=x)`	`hue="x"`

Use this when translating during the active learning.

Data first

In both R and Python, **tidy data** works best:

- One row = one observation
- One column = one variable

The DataFrames you built in Week 2 (filter, mutate, group_by) feed directly into seaborn. No extra reshaping if your data is already tidy.

➡ pandas from Class 2 feeds directly into plotting

Basic scatterplots

R (ggplot2):

```
ggplot(df, aes(x, y)) + geom_point()
```

Python (seaborn):

```
import seaborn as sns
sns.scatterplot(data=df, x="x", y="y")
plt.show()
```

Note: In scripts and some environments you need `plt.show()` to display the figure.

➡ Notebook Section 1

Aesthetics: color, size, style

R: `aes(color = group)`
Python: map variables to visual properties via arguments.

R (aes)	seaborn argument
color	`hue="group"`
shape	`style="group"`
size	`size="variable"`

```
sns.scatterplot(data=df, x="x", y="y", hue="group", size="size_col")
```

➡ Notebook Section 1

Histograms and density

ggplot2	seaborn
`geom_histogram()`	`sns.histplot()`
`geom_density()`	`sns.kdeplot()`

Histogram with density curve:

```
sns.histplot(data=df, x="x", kde=True)
plt.show()
```

➡ Notebook Section 2

Boxplots and violin plots

R: `geom_boxplot()`

Python: `sns.boxplot()`

```
sns.boxplot(data=df, x="group", y="y")
plt.show()
```

Violin plots show the full distribution (not just quartiles): `sns.violinplot(data=df, x="group", y="y")`.

➡ Notebook Section 2

Faceting

R: `facet_wrap(~ variable)` or `facet_grid()`
Python: `sns.relplot(..., col=...)` (or `row=`) for small multiples.

```
sns.relplot(data=df, x="x", y="y", col="group", kind="scatter")
plt.show()
```

`relplot` is the relational-plot factory; `kind="scatter"` gives scatter plots. Use `col=` (and optionally `row=`) for facets.

➡ Notebook Section 3

Figure control with matplotlib

Why matplotlib matters: seaborn draws on matplotlib axes. For titles, labels, limits, and saving, you use matplotlib.

```python
import matplotlib.pyplot as plt

sns.scatterplot(data=df, x="x", y="y")
plt.title("My title")
plt.xlabel("X axis label")
plt.ylabel("Y axis label")
plt.xlim(0, 100)
plt.savefig("plot.png", dpi=150)
plt.show()
```

➡ Notebook Section 4

Multiple layers: a key difference

ggplot2: Add layers with +.

```
p + geom_point() + geom_smooth()
```

Python: Call another plotting function; by default it draws on the current axes (overlay).

```
sns.scatterplot(data=df, x="x", y="y")
sns.regplot(data=df, x="x", y="y", scatter=False)  # line only
plt.show()
```

Order of calls = order of layers (first call = back, last = front).

Common pitfalls for R users

- **Expecting + layering** — In Python you overlay by calling another function (e.g. `sns.scatterplot` then `sns.regplot`), not with +.
- **Forgetting `plt.show()`** — In many environments figures don't appear until you call `plt.show()`.
- **Confusing figure vs axes** — matplotlib has "figure" (window/canvas) and "axes" (one plot). seaborn usually uses the "current" axes.
- **Over-customizing too early** — Get the data and plot type right first; then add titles and styling.

➡ Cheat Sheet Section 8

Active learning (15–20 min)

Translate this ggplot into seaborn.

R code:

```
ggplot(df, aes(x=size, y=price, color=type)) +
  geom_point() +
  facet_wrap(~ neighborhood)
```

Tasks:

1. **Predict** — What will the plot look like? (Scatter of size vs price; color by type; one panel per neighborhood.)
2. **Translate** — Write the seaborn version (`sns.relplot` with `col="neighborhood"`, `hue="type"`).
3. **Modify** — Add a title and axis labels with matplotlib.

➡ Notebook Section 5

Active learning – solution sketch

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.relplot(
    data=df, x="size", y="price", hue="type", col="neighborhood",
    kind="scatter"
)
plt.suptitle("Price vs size by type and neighborhood", y=1.02)
plt.show()
```

Note: With `relplot`, titles and labels may need `plt.suptitle()` or per-facet tweaks. Check the Week 3 Notebook for details.

Key takeaways

Today you can: explain the Python viz stack, create common plots in seaborn, customize with matplotlib, and translate ggplot2 to Python. When stuck, use the **R→Python Cheat Sheet, Section 8**.

1. **seaborn** handles most statistical graphics (scatter, hist, box, violin, facet) with a DataFrame-oriented API.
2. **matplotlib** handles fine control: titles, labels, limits, saving. Use it after seaborn for polish.
3. **Tidy data** from pandas simplifies plotting; same idea as in R.
4. **Layering** is done by calling another plot function on the same axes, not with +.

Looking ahead

Next week (Week 4): Modeling in Python

- statsmodels vs scikit-learn
- Inference vs prediction
- Bring your R→Python cheat sheet

Week 3's plots are how you'll visualize model results and explore data before modeling.

Thank you

Data 201 · Intro to Data Analysis in Python
Week 2-3 — Visualization in Python for R Users

Dr. Rebin Muhammad